Shipping Your Project

Autumn term 2026

Practical advice, not rules. The binding rules are in the Requirements document; nothing here changes them. If the two ever disagree, the Requirements win.

"Shipping" means handing over work that someone else can open, read and run without asking you a single question. Most marks lost on this project are lost here — not in the modelling.

Two things to do in the first week

Put your real name on your GitHub account. Requirement 4.13 asks for your full name in the account's profile name, and Requirement 4.14 says a submission that cannot be matched to a registered student is not graded. Most student accounts are called something else. Changing it takes ten seconds in Settings → Public profile, and it is the single cheapest way to avoid losing a grade you have earned.

Make the repository now, not in December. An empty repository with a README costs nothing and means the last week is about the work.

The test that matters

Before you submit, do this on a machine that is not the one you worked on:

1. `git clone` your own repository into an empty folder.
2. Follow your own README exactly as written, changing nothing.
3. See whether the results in your paper appear.

If step 3 fails, you have found what the marker would have found. Almost every problem below is caught by this one test. Cloning is the honest version of the test: it shows you only what you actually committed, not what happens to be sitting on your desk.

What the repository holds

```
paper.pdf                   the research paper
README.md                   how to reproduce, in order
requirements.txt            the packages needed
recording.mp4               or a link in the README, if the file is large
code/
  01_get_data.py            or .ipynb - one stage each
  02_clean.ipynb
  03_model.ipynb
  04_figures.ipynb
  helpers.py                anything used more than once
data/                       the data, or the script that fetches it
```

```
figures/                        what the paper shows
```

Numbering the files is not required, but it answers "what do I run first?" without anyone having to ask.

Use lowercase names with no spaces and no accents. `data final (2).csv` breaks on other people's machines; `prices_2020_2024.csv` does not.

One notebook per stage

Requirement 4.8 says no single file may contain the whole analysis. This is the requirement people most often ignore, so here is why it exists.

A 5000-line notebook that loads, cleans, models and plots cannot be checked. Nobody can run part of it, nobody can find the line that made a number, and nobody — including you, in January — can tell which cells were still live and which were abandoned three weeks ago. It also makes Requirement 6.1 impossible to satisfy: in one enormous file, no reader can tell which parts you wrote.

Split by stage instead: get the data, clean it, model it, make the figures. Four files. Each one starts by loading what the previous stage saved and ends by saving what the next stage needs.

Rules of thumb, not requirements:

- If a notebook takes more than a couple of minutes to scroll through, it is doing more than one job.
- If you have copied a function into a second notebook, it belongs in `helpers.py` (Requirement 4.9).
- If a cell only works when you run the cells above it in a particular order that is not top to bottom, the notebook is broken. Restart the kernel and run all, before you commit.

Keep the repository clean

Requirement 4.12 excludes caches, checkpoints, virtual environments and editor settings. A `.gitignore` handles all of it:

```
__pycache__/
.ipynb_checkpoints/
.venv/
.DS_Store
.vscode/
```

Add it in the first week and you never think about it again.

Working with the repository

- **Commit as you go.** A commit a day is a record of your work. One enormous commit on the last evening is a record of nothing, and it is also when things go wrong.
- **What counts is the last commit before the deadline** (Requirement 4.12). Committing is not the same as pushing: run `git push` and then look at the repository page in your browser to confirm your work is actually there.
- **Do not commit very large files.** GitHub rejects anything over 100 MB and gets slow well before that. If your data is large, Requirement 4.15 lets you commit the download script instead.
- **Use a `.gitignore`** for caches, checkpoints and virtual environments (`__pycache__/`, `.ipynb_checkpoints/`, `.venv/`). They are noise, and they make the repository heavy.

- **Never commit a key or a password.** Deleting it in a later commit does not remove it: it stays in the history. If it happens, tell us and revoke the key.

The README

Half a page is plenty. In this order:

- what the project does, in two sentences;
- what to install (`pip install -r requirements.txt` is the whole answer, if you write the requirements file);
- what to run, in the order to run it;
- how long it takes, and what it produces;
- where the data came from.

Write it as instructions to a stranger, because that is who reads it.

Code

- **A script that runs top to bottom beats clever code that needs explaining.** You are graded on evidence that you can apply the methods, not on style.
- **Set the random seed.** If your numbers change every run, nobody can check anything in your paper.
- **No absolute paths.** `/Users/anna/Desktop/thesis/data.csv` exists on exactly one computer in the world. Use `data/prices.csv`.
- **Restart the notebook and run it once, in order, before submitting.** Notebooks that work only if cells are run out of order are the most common reproducibility failure there is.
- **Keep secrets out.** No API keys in the code. If your project needs one, say in the README which key is needed and where to put it.

Data

- Small and shareable: include it.
- Large or licensed: include the download script and say plainly what the licence permits. Requirement 4.15 allows this.
- Say where every data set came from and what you did to clean it. "I dropped 412 rows with missing prices" is a sentence the marker wants to read; silence about it is not.

The paper

- **Open the template on day one**, not in December: Template for SIAM Journals → **Open as Template**. It creates an Overleaf project in your account. Nothing to download, nothing to install, and it compiles in the browser. Writing into the template from the first paragraph costs nothing; reformatting a finished paper into it costs an evening.
- If you have never used LaTeX: you do not need to learn it properly. Replace the title, the author, and the section bodies. The template already handles the rest.
- Overleaf's free tier is enough for this project. If you would rather write elsewhere, that is allowed — Requirement 4.4 asks for the output format, not the tool.
- Every figure needs axis labels, units, and a caption saying what to look at.
- Report what you actually ran. If a method failed, that belongs in the paper — Requirement 7.3 lets an honest negative result score full marks.

The recording

- Fifteen minutes, screen plus voice. A Zoom recording of yourself is fine.
- A structure that fits the time: the question (2 min), the data (2 min), the method (4 min), the results (5 min), what you would do next (2 min).
- Watch the first minute back before submitting. Inaudible sound is the usual failure, and it cannot be fixed after the deadline.
- Say your name at the start.

Compute: what you get, and scoping to it

You get a Nuvolos workspace, free, for the whole course. That is the only compute the University provides for this project — there are no cloud credits, no GPU allocation, and no licences beyond what is already in the workspace.

The workspace is modest and deliberately so: it is enough for the methods this course teaches, applied to data of a sensible size. Almost every good project fits in it comfortably.

Scope to it at proposal stage, not in December. If your idea needs to fine-tune a large model or grid-search for two days, it is out of scope (Requirement 4.22), and the proposal is where we say so — which is the whole point of asking for a proposal. Bring the question then, not after you have spent three weeks on it.

Ways a project shrinks to fit without becoming trivial:

- take a sample of the data rather than all of it, and say so in the paper;
- use fewer, better-chosen parameter settings instead of a large search;
- pick the smaller model. A well-understood small model beats a large one you could not inspect.

If you use your own compute, you may — at your own cost, and it earns you nothing. Requirement 7.2 assesses the work, not the hardware. Two practical obligations come with it: say in the README what you ran it on and roughly how long it took (Requirement 4.24), and if the result cannot be reproduced in the workspace, commit the code, the saved output, and a note of what produced it (Requirement 4.25). A number nobody can trace back to code is not evidence.

And note Requirement 4.26: a machine you rented failing, or running out of credit, is not a reason for an extension. The workspace you were given does not have that failure mode.

Code you did not write

Section 6 of the Requirements is the one people get wrong, so here is the short version.

You do not attribute libraries. `import pandas` is using a tool.

You do attribute anything you did not write yourself: a function from Stack Overflow, a plotting recipe from a blog, a chunk of a repository, a replication package from a paper. One comment where it appears is enough:

```
# adapted from https://github.com/someone/repo, MIT licence
def rolling_sharpe(returns, window):
    ...
```

Then list it in the appendix with the licence. That is the whole job, and it takes about a minute.

Where it gets awkward: assistants. When ChatGPT writes a function, it sometimes

reproduces code that already exists somewhere, and does not tell you. You cannot check this, and you are not expected to. What you can do is say the function came from an assistant and what you asked for. If someone later recognises the code, that declaration is the difference between an honest mistake and a misconduct case. It is a minute's work buying a great deal of protection.

The test to apply: for every block of code, can you say where it came from? If the answer is "no idea, it has been in the folder for weeks", that block cannot be attributed — and Requirement 6.7 says it should not be submitted. Rewrite it or find the source again.

Attribution never costs you marks. Requirement 6.9 is explicit: what is assessed is what you built with the code, and the offence is the silence, not the borrowing.

Declaring your tools

Requirement 5 asks which tools you used and what for. A few honest lines:

> ChatGPT: drafted the plotting code in `figures.py`; rewrote three paragraphs of the introduction for clarity. All statistical results are my own work and were checked by hand.

Nobody is penalised for using assistants — the course teaches you to. The penalty is for not declaring them, and for being unable to explain what you handed in.

Submitting: the commit identifier

Requirement 4.17 asks you to email the repository URL **and** the commit identifier — the SHA. That number is what makes your submission fixed: it names one exact state of your work, so nothing you do afterwards can change what is graded, and nothing anyone else does can either.

To get it, once everything is committed and pushed:

```
git log -1 --format=%H
```

That prints 40 characters. Copy the whole thing into the email.

Check it is really on GitHub before you send: open your repository in a browser and find that commit. `git push` failing quietly, or a commit sitting on a branch that is not the default one, is the usual way this goes wrong — and Requirement 4.19 treats a commit that cannot be found as a submission not made.

You can keep working after you send the email. It changes nothing: the commit you named is the one that is graded. What you must not do is delete the repository, make it private to us, or rewrite the history — that removes the commit you submitted.

The week before the deadline

- Run the reproduction test above. Now, not on the last evening.
- Check the page count against Requirement 4.3.
- Check the eight required sections are present and in order.
- Send the submission email a day early with the SHA of what you have. If you improve the work afterwards, send a second email with a new SHA before the deadline; the last one before the deadline is the one that counts. An email you have already sent is worth more than a perfect one you were about to send.